Build Report

MovInBuddy: Building an Android App with Claude

A move-in inventory app, built end-to-end in conversation — Gradle builds, an emulator, and a PDF report generator, all driven without writing code by hand.

Tool : Claude Code (Sonnet 5)

Platform : Android · Pixel 10a emulator

Stack:Kotlin, no backend

01 Business Driver

Residential move-ins still run on a paper — or at best a static, unfillable PDF — condition form. In practice, that process breaks down in a few predictable ways:

- **Photos get skipped.** Walking a full property with a clipboard is tedious, so people photograph the one or two "obviously bad" spots and skip everything else — including the damage they'll get blamed for later.
- **Photos exist but prove nothing.** Even when photos are taken, they land in a generic camera roll — "IMG_4821.jpg" — with no structured link back to which line item, which room, or which specific fixture they document.
- **Notes and photos live in different places.** A paper form in a drawer and a phone full of unlabeled photos don't cross-reference each other, so reconstructing "what did I actually note about the guest bathroom" months later means manual detective work.
- **Disputes come down to who kept better records.** When a move-out deposit is contested, the burden of proof falls on whoever can produce dated, specific, verifiable documentation — and a paper form with no photo traceability rarely holds up.

MovInBuddy's answer is to make the record structurally impossible to disconnect: every comment and every photo is captured against the same line item, in the same action, and the exported PDF stamps each appendix photo with a reference code (e.g. `S6.5-P1`) that ties it straight back to its row in the inventory table. Documenting the property and being able to prove it later become the same step.

02

Project Overview

MovInBuddy is an Android app for tenants and people moving into a new home. It replaces the paper "walk the property with a clipboard" ritual with a guided digital inventory: define sections of the home (Exterior, Kitchen, Bedroom, etc.), check off line items in each section, attach comments and photos to any item, and generate a single PDF report at the end — a residential inventory-and-condition form followed by a photo appendix, with tenant and landlord signature lines.

Core flow implemented:

1. **Sections & items** — a skeleton home screen lists sections (mailboxes, fences, pool, kitchen, bedrooms, etc.) with a "+" to duplicate section types for homes with multiple bedrooms or bathrooms.
2. **Item detail** — tapping an item opens a screen to type a condition comment and attach multiple photos, via camera capture or the device gallery.
3. **Review screen** — an in-app screen listing every section/item with its comment and photo count, standing in for a "shareable review link" kept deliberately local rather than cloud-hosted.
4. **Finish → PDF** — a multi-page PDF styled like a standard inventory/condition form: a table per section, every item listed (blank where there's no comment), comments in blue, a photo appendix traceable back to section/item by reference code, and a signatures block for tenant and landlord.
5. **Utility features** — Clear All Data, a custom adaptive app icon, and gallery-picker demo photos for testing without a real camera.

03

Datasets Used

None — this project used no external datasets. It's a self-contained mobile app with local JSON-based storage, no database, no cloud backend. The only "data" involved were:

- **Section/item templates** — a hand-authored list of roughly a dozen standard property sections and their line items, written to match the structure of a real-world residential lease inventory form without copying its protected text.
- **A reference PDF** — a Texas Association of Realtors residential lease inventory form, used only as a structural reference for the app's own PDF layout and its signature section.
- **Synthetic demo photos** — with no real camera on the emulator, ~35 placeholder JPEGs were generated programmatically to populate the gallery picker: labeled room-scene illustrations, then a themed "problem" set (a broken tile, a corroded faucet, a hole in a bedroom door) for a realistic demo — plus one deliberately funny Easter egg photo for demo flair.

04

Prompts Used During Vibe Coding

The exact prompts, in order, typos included. The first was deliberately trivial — not a feature request, but a check that the Android toolchain (SDK, emulator, Gradle) was actually installed and working before attempting the real app.

> **01 — environment check**
>
> ```
> Build a simple "hello World" android mobile app for pixel 10 a
> ```
>
> **02 — kickoff: goal + MVPs**
>
> ```
> Now let us build the Real good app, with the goal and Mvps listed
> below. Read the goals and MVP mentioned below and brainstorm with me
> on what you think, whether any similar app already exists. Proceed
> ```

for building once I confirm to go ahead. Meanwhile also let me know if there is any similar app that already exists to do this today.

The goal is to build an Android app that is useful for tenants or people who are moving to new homes. They want to quickly take a digitized inventory condition form, click pictures, associate with each of the items under a section, and then create a PDF file that they can submit to the landlords. The pdf will have the Residential form with all the comments entered by the user followed by an appendix that will contain pictures that were taken.

Here are the MVPs.

1. First build a skeleton UI flow or UI navigation panels. Users should be able to enter each section and each section should have the list of items mentioned under each section in the form. For example:
- mailboxes
- fences and gates
- pool
- spa
- equipment
The user should be able to navigate back to the main screen, the home screen. In cases where the user wants to add multiple rooms or multiple bathrooms, there should be a plus button to add an additional section for that particular home.

2. When the user enters a section, they should be able to select each line item within the section. For each line item they should be able to enter comments and take a number of pictures. The pictures taken should be saved in a way that is mapped to the individual line item within a section.

3. When the user is done with entering comments and clicking pictures, there should be a link provided in the tool where the user can go to that link, check all the pictures clicked, and also see the comments that one entered for each of those line items.

4. Once the user is done entering each section and clicking the pictures, adding comments to each of the sections, the user should be able to navigate back to the home page and the Done button or Finish button. When this is done a PDF should be created that is similar to the residential lease and inventory condition form. This should be followed by a set of pictures that were taken and provide some sort of a correlation for each of the sections and the section items in the PDF. the pictures in the end of the residential inventory form should be traceable with respect to each line item in

the section, the comments provided, and the set of pictures that are associated with that section.

5. There should be an option to clear that allows the user to start everything from scratch
6. Along with clicking pictures, user should have option to select pictures from phone
7. Once the user clicks Finish button, there should be an option to view the final pdf
8. Add a relevant nice icon to the project
9. The comment that appears once the user moves back to the home screen should be an indifferent color like blue, generally the comments should appear in blue where they are.

Add some pictures may be 2 for each of the sections that user can pick, select the item randomly for pictures

03 — form fidelity

The final PDF generated should be exactly like the Texas realtors format. If there are entries with no comments , leave them blank. Pictures with traceability back to section/items in section should follow after the original inventory form.

04 — clarifying answer

Close structural math is good, but i need all the sections and items listed down blank if there are not comments

05 — environment

restart the android virtual device

06 — bug report

Why is the emulator screen showing blank i don't see the home screen and none of the apps visible

07 — demo data

```
can you clear all the pictures in the emulated device and add few
dummy pictures for a fictitious property that i can select for my
testing
```

08 — bug report

```
restart the Moveinbuddy application on virtual device, it is hung
```

09 — quality + bug report

```
Instead of text pictures, can generate real world like pictures,
with comments as watermark and why are the pictures looking weird
like enlarged and blurred once they are selected in the app. fix it.
```

10 — environment

```
on the virtual device i have zoomed in, all the screens are
appearing large. how to zoom out
```

11 — feature request

```
I noted that the pdf does not contain the Tenant and owners
signature part. refer the flease_inventory_form and add similar
signature section
```

12 — bug report + creative ask

```
The pictures visible when i select through the app and after
selection are different. Check and fix it. My request is please look
at each section and pick any 2 random items under each section and
create a dummy image so that I can select it for the demo. name the
pictures according to the problem. For example, flooring provides a
broken tile picture, or broken faucet, or a bedroom door with a hole
or to make it funny. put a spider man hanging upside down in a room.
when demoing i can say spider man in a room and add that picture
```

Iterations Tried

These are the refinement loops on the requester's side — places where a first ask didn't land exactly right and needed a follow-up prompt to converge, rather than the internal code changes made along the way.

PDF format: broad ask → precise constraint

The first request was "make the PDF exactly like the Texas Realtors format." When asked to choose between a literal copy (licensing risk) or a close structural match, the follow-up sharpened the actual requirement: keep the structure, but list every section/item even when blank rather than omitting empty rows.

Demo photos: three separate asks to get right

Round 1 asked for generic dummy pictures for a fictitious property. Round 2 came back after seeing them: make them look more real, add comments as a watermark, and fix why they render blurred/enlarged in-app. Round 3 asked for something more specific still — 2 photos per section named after an actual problem (broken tile, broken faucet, hole in a door), plus a fun Easter-egg photo for demos.

Environment troubleshooting: several short, reactive prompts

Across the session, emulator issues were reported one at a time as they appeared — "restart the virtual device," then "the screen is showing blank," then "the app is hung, restart it," then "everything is zoomed in, how do I zoom out" — each a separate turn rather than one bundled description of the environment misbehaving.

Signature section: caught after the fact

The PDF was already being generated and reviewed before it was noticed that it had no tenant/landlord signature section; the fix request pointed directly at the reference lease-inventory form's own signature page to match against.

Photo picker mismatch: reported, then scoped into a bigger ask

What started as "the picture I select doesn't match what shows up in the app" was folded into a larger, more specific follow-up in the same message: regenerate all demo photos per-section, name them after the specific problem they depict, and include a joke photo for demos.

06

Learnings & Observations

Token-cost lesson:
A popup came up mid-session requesting an Adobe tool installation. Rather than reusing context already established earlier in the conversation, a fresh request was made to "install the tool" — which alone burned about 1.2K tokens working out the install steps from scratch.
The cheaper move would have been to point Claude back at the earlier chat message where the same situation had already been resolved, and simply say "do that again" — letting it reuse the already-worked-out steps instead of re-deriving them from zero.
Takeaway: when a familiar prompt or popup reappears, reference the prior message instead of re-explaining the ask — it's both faster and meaningfully cheaper.

Two things worth doing differently next time :
1. Bundle environment symptoms into one message instead of reporting them as they appear. Emulator trouble was flagged one turn at a time — restart, then "screen is blank," then "app is hung," then "everything is zoomed in" — and each one kicked off a fresh diagnostic pass. Describing the whole sequence in a single message ("after the restart the screen went blank, and on the next launch it looked zoomed in") would have let the root cause get found in one pass instead of several.
2. Describe what's actually happening, not the fix being requested. A few prompts named a presumed fix rather than the symptom — "restart the app, it's hung" when the real cause was a leftover camera-result screen stuck in the background; "how do I zoom out" when it turned out to be the PDF viewer's own zoom state, unrelated to the emulator. Leading with the raw observation gives more room to diagnose the actual cause, rather than performing the requested fix and landing right back at the same problem.
Confirm big scope changes before building. Before writing any code for the "real" app, Claude was asked to brainstorm competing apps and flag concerns — like a copyrighted form format

versus a generic one — before implementing, catching a licensing risk early rather than after the PDF was built.